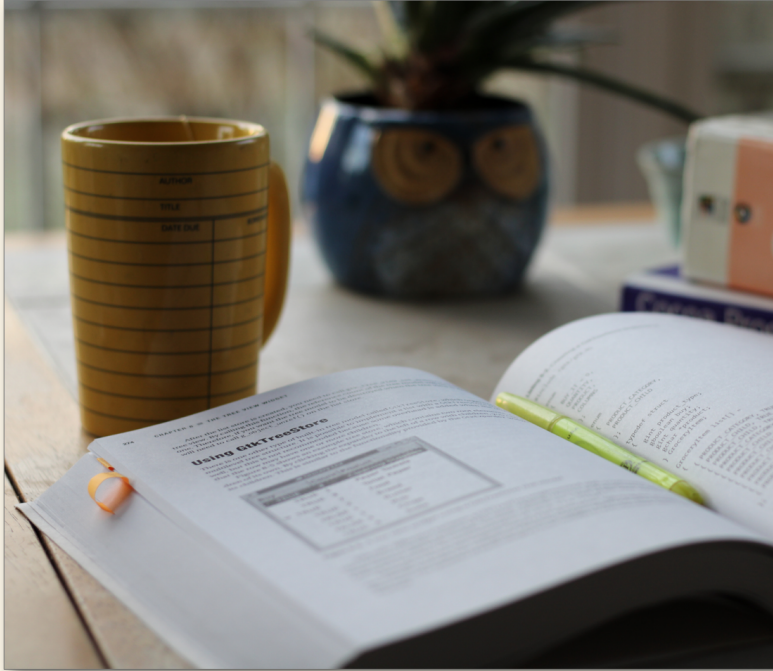


Readability Studio 2026

System Administrator Manual

READABILITY ANALYSIS SOFTWARE



Metric	Status
Open Issues	Issues 0 open
Open Pull Requests	pull requests 0 open

Test Statuses	
Platforms	Result
Linux	Linux Build & Unit Tests passing
macOS	macOS Build & Unit Tests passing
Windows	Windows Build & Unit Tests passing

Code Analyses	Result
cppcheck	cppcheck passing
clang-tidy	clang-tidy passing
MS PRefast	Microsoft C++ Code Analysis passing
Quneiform	quneiform passing

Documentation & Formatting	Result
Doxygen	doxygen passing
Spellcheck	Spell Check passing
Profanity Check	Profanity Check passing
clang-format	test-clang-format passing
UNIX Newlines	Check LF line endings passing





Readability Studio 2026

System Administrator Manual

Blake Madden

Eclipse Foundation AISBL

Brussels

Ottawa

Darmstadt

Readability Studio 2026

System Administrator Manual

Copyright © 2026 Blake Madden

Eclipse Readability Studio® is a trademark of Eclipse Foundation AISBL

All rights reserved.

Published in the United States

This book is distributed under a Creative Commons Attribution-Sharealike 4.0 License.



That means you are free:

- To Share – copy and redistribute the material in any medium or format.
- To Adapt – remix, transform, and build upon the material.

The licensor cannot revoke these freedoms as long as you follow the license terms:

- Attribution – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- Share Alike – If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions —You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Table of contents

Preface	1
---------------	---

I

Administration

1	Microsoft® Windows 	5
	Installing Locally	5
	Installing Remotely	5
	Uninstalling	5
	Updating	6
	Command Line Options	6
2	macOS 	7
	Installing	7
	Uninstalling	7
	Updating	7
3	Linux 	9
	Installing	9
	Uninstalling	9
	Updating	9
4	Telemetry	11
	Log File	11
	Connectivity	12
5	Reporting Issues	13

II

Building from Source

6	Build System Overview	17
	Requirements	17
	Build Configurations	17
	wxWidgets Integration	18
	Submodules	18
	Versioning and Configured Files	18
	Platform-Specific Compilation	18
	Unit Testing	19
	Resource Bundling and Staging	19
	Help and Documentation Targets	20
	Installer Packaging	20
7	Microsoft® Windows 	21
	Download wxWidgets	22
	Build <i>Readability Studio</i>	22
	Build the installer	22
8	macOS 	23
	Download wxWidgets	25
	Build <i>Readability Studio</i>	25
	Code sign	26
	Build the DMG image	27
9	Linux 	29
	Download wxWidgets	30
	Build <i>Readability Studio</i>	30
	Build the AppImage	30

III

Contributing

10	Getting Involved	33
	Report a Bug or Request a Feature	33
	Contribute Code, Documentation, or Other Assets	34
11	Coding Style Guide	35
	Naming Conventions	35
	Styling	36
	Bitmask Enumerations	36
	Best Practices	36
	i18n Guidelines	36
12	Code Structure	39
13	Coding Procedures	41
	Adding a New Option	41
	13.1 Adding a New Option to a Graph's Ribbon	42
	Adding a New Test	43
	Adding a New Familiar Word List	44

Adding a New Custom-Test Formula Function	47
Adding a New Stipple Shape	47
Adding a New Color Scheme	47
13.2 Adding New Files	48
14 Documentation	49
Style Guide	49
Editing	49
Screenshots	50
Building	51
15 Resources	53
Art Assets	53
Word Lists	54



Preface

This book is a guide to building, installing, and maintaining the software product *Readability Studio*. System administrators can follow this guide for deploying installations and updates of *Readability Studio*. Additionally, instructions are provided for system administrators and individual users for how to build the program from source and optionally install it.

Readability Studio is available for Microsoft® Windows , macOS , and Linux .



Administration

1	Microsoft® Windows 	5
2	macOS 	7
3	Linux 	9
4	Telemetry	11
5	Reporting Issues	13



1. Microsoft® Windows

Requirements

64-bit Windows 10 (or higher)
x86_64 processor
2 GB of RAM (4 GB recommended)

The *Microsoft Visual C++ runtime* is also required. This is normally managed via *Windows Update*, but can also be installed manually. To install, download and run the latest [Microsoft Visual C++ Redistributable](#) installer.

The *Microsoft Edge WebView2 Runtime* is also required. This is pre-installed on Windows 11 and is normally managed via *Windows Update*, but can also be installed manually. To install, download and run the latest [Microsoft Edge WebView2 Runtime](#) installer.

Installing Locally

Run the installer ("ers-setup.exe") as an administrator. You will be prompted for a user name, where to install the program, and which components to install. After answering these prompts, continue the installer to completion.

Installing Remotely

Run the installer ("ers-setup.exe") via a remote tool, such as *Windows® SCCM (System Center Configuration Manager)*. Refer to the end of this chapter for available command line options.

Uninstalling

The program can be uninstalled via the system's Installed apps panel or from the command line.

In the installation folder of *Readability Studio*, there will be file named similar to "unins???.exe." System Admins can run this program remotely to uninstall the program from workstations.

 Tip

"unins???.exe" can take parameters, such as /VERYSILENT.

Updating

To upgrade an installation, run the installer ("ers-setup.exe") as an administrator and follow the prompts. If there is a previous installation of *Readability Studio*, then the installer will update it.

 Note

If updating a 32-bit edition of *Readability Studio* (version 2021 or earlier), you will first need to uninstall the program. The 64-bit editions default to the 64-bit  Program Files folder, while earlier editions were installed to the  Program Files (x86) folder. Uninstalling any 32-bit editions will ensure that you won't have multiple installations after updating.

Command Line Options

The installer uses *Inno Setup*, which provides support for silent install and other options. (This applies to installing, uninstalling, and upgrading.) To perform a silent install, pass the command line arguments /SILENT or /VERYSILENT. For example, the following will run a silent install that will not restart the computer and write its progress to a log file:

```
ers-setup.exe /SILENT /NORESTART /LOG
```

Available command line options include:

Option	Purpose
/SILENT	Skips user prompts, keeping the default options.
/VERYSILENT	Same as '/SILENT', but also hides the installer's window.
/LOG	Log the installer's progress to a temp file.
/LOG=filename	Same as '/LOG', but writes to the provided filename.
/NORESTART	Do not reboot after installation, even if necessary.
/CLOSEAPPLICATIONS	(Updater only) Close the program if open during the update.
/DIR=foldername	(Installer and updater only) Where to install (or update) the program. (This will override the default installation path.)



2. macOS

Requirements

macOS 13.3 (or higher)
Apple Silicon or Intel processor
2 GB of RAM (4 GB recommended)

Installing

Open "ReadabilityStudio.dmg" and drag-and-drop the program into the "Applications" folder. This will copy "Readability Studio.app" (the self-contained program) into where the rest of your applications are installed.

Uninstalling

From the "Applications" folder, drag "Readability Studio.app" into the Trash bin.

Updating

Open "ReadabilityStudio.dmg" and drag-and-drop the program into the "Applications" folder, replacing the existing copy.

3. Linux

Requirements

x86_64 processor

2 GB of RAM (4 GB recommended)

Installing

Make “Readability_Studio-x86_64.AppImage” executable. This can be done with the following syntax:

```
chmod u+x <AppImage File>
```

This can also be done by right-clicking the file, selecting Properties Permissions and checking *Allow executing file as program*.

This only needs to be done once. Now “Readability_Studio-x86_64.AppImage” can be run directly to launch the program.

Note

An AppImage is a portable Linux app with all dependencies included, ready to run without installation.

Uninstalling

Drag “Readability_Studio-x86_64.AppImage” into the Trash.

Updating

Replace your existing AppImage with the latest version. Make the new AppImage executable and then it will be ready to run.



4. Telemetry

Readability Studio does not transmit or share user information. If a user needs to share usage information generated by the program, they must do it explicitly.

The program does not require an internet connection, except for when users instruct it to analyze or download webpages. *Readability Studio* does not “phone home” and does not transmit your documents or results.

All analyses are performed on your local machine and do not require connecting to external services.

Log File

While it is running, *Readability Studio* logs various diagnostic information. This information can provide details such as:

- The location of where the program is loading its license and settings files
- Parsing errors encountered while reading documents (e.g., malformed HTML files)
- Warnings and errors encountered while performing analyses
- The paths of documents being analyzed (verbose mode only)
- Information about the operating system (e.g., OS version, default font, etc.)
- Version information about various libraries used by the program

Except for file paths, no personal identifying information (PII) is recorded.

This information is only saved to a file in the user’s local temp folder. It is not transmitted to any external service and is only intended for diagnosing issues with the software. This file can be optionally included in crash reports generated by the program. These crash reports must, however, be explicitly sent to the vendor by the user.

💡 Tip

To view this log report, go to the Tools tab on the ribbon, then click Log Report under the Tools & Settings section. From the Log Report dialog, you can review, save, and print this information.

Connectivity

Readability Studio only connects to the internet when it is instructed by the user to analyze or download a webpage.

The program has two interfaces for internet connectivity. The first is that it can analyze a webpage provided by the user. The second is its Web Harvester feature, which can crawl a user-provided website, where it will read and download its links. When connecting to these websites, the program will only send basic page request information; user's personal information is not transmitted.



5. Reporting Issues

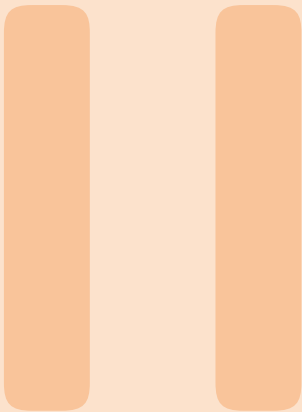
To report an issue or request a feature, visit <https://github.com/eclipse-ers/ReadabilityStudio/issues>.

When entering an issue, please include the following:

- A description of the issue or feature request
- If an issue, step-by-step instructions for how to reproduce it
- If a feature request, explain how you would like for the feature to work and a possible use case
- The version of *Readability Studio* and your operating system information (this can be copied from the program's About dialog)
- Any supporting documents or screenshots (please do not include sensitive or confidential data)

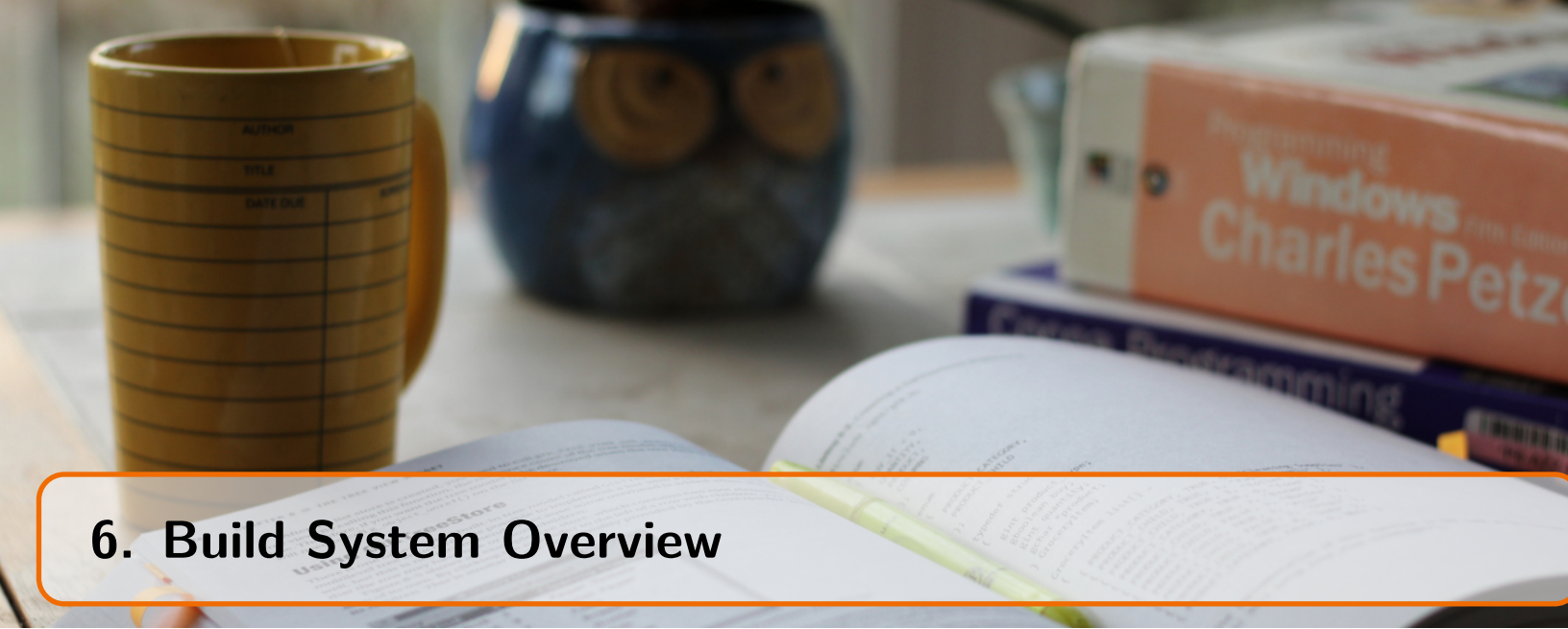
Warning

Any issue that you report will be publicly available and can be read by anyone. Do not include any personal or proprietary information or files in your report.



Building from Source

6	Build System Overview	17
7	Microsoft® Windows 	21
8	macOS 	23
9	Linux 	29



6. Build System Overview

Readability Studio uses a single *CMake* build script (`CMakeLists.txt`) to build the application across all supported platforms. This chapter provides a general overview of the build process, detailing the requirements, options, and platform-specific behavior of this script.

Requirements

The build system requires the following minimum versions:

Requirement	Version
<i>CMake</i>	3.26
C++ standard	C++20
<i>GCC</i> (🐉)	12.2.1
<i>AppleClang</i> (🍏)	15.3
Microsoft <i>Visual Studio</i> (🖥️)	2022 (toolset 19.30)
<i>wxWidgets</i>	3.3.3

Build Configurations

If no build type is specified, the script defaults to Debug. Two build types are supported: Debug and Release.

The following options are available:

- `ENABLE_SANITIZERS`: enables *AddressSanitizer* and *UndefinedBehaviorSanitizer* (ignored in release builds)
- `ALLOW_DIRTY_SUBMODULES`: allows building even if Git submodules have local changes. This is always ON for debug builds and defaults to OFF for release builds.

In debug builds, `wxDEBUG_LEVEL` is set to 1; in release builds, it is set to 0. The script verifies that `wxBUILD_DEBUG_LEVEL` and `wxDEBUG_LEVEL` are consistent, and will halt with an error if they are mismatched.

wxWidgets Integration

wxWidgets is discovered by searching parent directories above the project folder. It is built as a static library (`wxBUILD_SHARED OFF`).

The following *wxWidgets* libraries are linked:

- core, base, xrc, propgrid, xml, aui, ribbon, qa, stc, net, richtext

Submodules

The project depends on the following Git submodules:

- *Catch2*: testing framework
- *Lua*: scripting language
- *OleanderStemmingLibrary*: text stemming
- *TinyExpr++*: formula parsing engine
- *TinyXML2*: XML parser
- *Wisteria-Dataviz*: graphics library (includes *CRCpp*, *easyexif*, *utfcpp*, *wxPdfDocument*, *wxSimpleJSON*, and *wxStartPage*)

During configuration, the build script queries each submodule for its commit hash, remote origin, and version tag. These are written into a generated SBOM header (`src/app/sbom.h`) for traceability.

Versioning and Configured Files

The project version is defined via `project(readstudio VERSION ...)` and is broken into major, minor, patch, and tweak components. A display version string is built that omits trailing zero components (e.g., “2026.1” instead of “2026.1.0.0”).

The current Git commit hash is embedded as the build version.

During configuration, `configure_file()` generates the following from their `.in` templates:

- `src/app/version.h` — compiled version information
- `current-version.txt` — plain-text version for remote update checks
- `citations.bib`, `citations.mla`, `citations.apa` — citation files
- `resource.h` — Windows resource header
- `docs/_variables.yml` — *Quarto* documentation variables
- `docs/sysadmin/build-commands.txt` — build command reference
- `docs/sysadmin/windows-silent.txt` — silent install parameters
- `docs/readability-studio-manual/_quarto.yml` — manual *Quarto* configuration

On Linux (🐧), a “.desktop” file is also generated. On Windows (🪟), *Inno Setup* installer scripts are generated.

Platform-Specific Compilation

6.0.1 Windows (MSVC) 🪟

- Definitions: `__WXMSW__`, `CRCPP_USE_CPP11`, `_CRT_SECURE_NO_WARNINGS`, `_UNICODE`
- Debug adds: `__WXDEBUG__`, `ENABLE_PROFILING`, sanitizer definitions

- Key flags: `/Zc:__cplusplus, /MP, /W3, /WX, /openmp, /permissive-, /sdl, /utf-8`
- Release optimization: `/O2`

6.0.2 Windows (MinGW/MSYS)

- Same definitions as *MSVC*
- GCC-style flags: `-Wall, -Wextra, -Wpedantic, -Wshadow, -Werror`
- Security hardening: `-fstack-protector-strong, -D_FORTIFY_SOURCE=2`
- Release optimization: `-O2`

6.0.3 macOS

- Definitions: `__WXOSX__, CRCPP_USE_CPP11`
- Universal binary: `arm64` and `x86_64` architectures
- Deployment target: `13.3`
- Hardened runtime enabled
- GCC flags: `-Wall, -Wextra, -Wpedantic, -Wshadow, -Werror`

6.0.4 Linux

- Definitions: `__WXGTK__, CRCPP_USE_CPP11`
- Requires *GTK 3*
- *OpenMP* enabled for multithreading; *Intel TBB* linked if available
- Dead code stripped in release builds (`-s`)
- Symbol visibility set to hidden
- *clang* flags: `-Wall, -Wextra, -Wpedantic, -Wshadow, -Werror`
- Linker hardening: `-Wl,-z,relro,-z,now`

Unit Testing

When `BUILD_TESTING` is enabled, the build script runs all unit test suites during the configure step. Test results are written in *JUnit* XML format to the `test-results` directory.

The following test suites are executed:

Test Suite	Runner
<i>OleanderStemmingLibrary</i>	<i>OleanderStemTestRunner</i>
<i>TinyExpr++</i>	<i>TETestRunner</i>
<i>Wisteria-Dataviz</i>	<i>WisteriaTestRunner</i>
<i>Wisteria-Dataviz (GUI)</i>	<i>WisteriaGuiTestRunner</i>
<i>Readability Studio</i>	<i>RSTestRunner</i>
<i>Readability Studio (GUI)</i>	<i>RSGuiTestRunner</i>

Resource Bundling and Staging

After building, the following resources are bundled and copied to the build output and installer staging directories:

- `res.wad`: images and menu resources (zipped)

- `words.wad`: word lists (zipped)
- Documentation folders (user manual, cheat sheets, test reference, release notes, sysadmin manual, API docs, Lua reference)
- Translation files (e.g., Spanish)
- License and citation files

Help and Documentation Targets

The build script provides additional targets beyond the main application:

- `manuals`: builds the documentation using *R* and *Quarto* (requires *R* to be installed)
- `doxygen-docs`: generates API documentation with *Doxygen* (included with ALL if *Doxygen* is found)
- `g11n-dashboard`: globalization dashboard showing translation statuses
- `docs-dashboard`: documentation coverage and completion dashboard
- `tests-dashboard`: tests and quality dashboard

Installer Packaging

In release builds, the binary and resources are automatically staged for installer creation:

- Windows (🪟): the executable is copied to `installers/windows/release`.
An *Inno Setup* script is configured for building the installer.
- Linux (🐧): the executable is copied to the Applmage staging directory at `installers/unix/AppDir/usr/bin`.
A `.desktop` file and application icon are also staged.
- macOS (🍏): the application bundle is copied to `installers/macos/release`.
The `Info.plist` is configured from its template.



7. Microsoft® Windows

Install the following tools to build *Readability Studio*

Visual Studio 2022 or higher

Inno Setup

R from CRAN (optional)

Quarto (optional)

POEdit (optional)

Download wxWidgets

- Open *Visual Studio* and select Clone a Repository
 - Enter ["https://github.com/wxWidgets/wxWidgets.git"](https://github.com/wxWidgets/wxWidgets.git) and clone (i.e., download) it

Build *Readability Studio*

- From *Visual Studio*, select Clone a Repository again
 - Enter ["https://github.com/eclipse-ers/ReadabilityStudio.git"](https://github.com/eclipse-ers/ReadabilityStudio.git) and clone it to the same level as the "wxWidgets" folder
- Once the "ReadabilityStudio" folder is cloned, open it in *Visual Studio*
- Select the "Visual Studio 2022 Release" configuration and build it
- Optionally, the "manuals" target can also be ran to rebuild the documentation

💡 Tip

To create a debug build, select the "Visual Studio 2022 Debug" configuration. Additionally, selecting "Visual Studio 2022 Debug w/ ASAN" will enable AddressSanitizer. To allow building with uncommitted changes in Git submodules, open your "CMakePresets.json" file and add the following to your chosen profile:

```
"cacheVariables": {
  "ALLOW_DIRTY_SUBMODULES": "ON"
}
```

Build the installer

💡 Tip

The following steps are only required if you are distributing builds outside CI, your local machine, or Eclipse's infrastructure.

📝 Note

If you build the "Release" version of *Readability Studio*, then the CMake build process will copy "readstudio.exe" into `installers >> windows >> release`.

- Go to `installers >> windows`
- Digitally sign "release/readstudio.exe"
- Open "readstudio.iss" in *Inno Setup* and build the installer
 - The installer will be placed in "output/ers-setup.exe"
- Digitally sign the installer



8. macOS 🍏

Install the following tools to build *Readability Studio*

XCode 15.3 or higher
CMake 3.26 or higher
create-dmg
Homebrew
R from CRAN (optional)
Quarto (optional)
POEdit (optional)

To install home-brew:

```
/bin/bash -c \  
"$(curl -fsSL \  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

To install *CMake* and *create-dmg*:

```
brew install cmake  
brew install create-dmg
```

If you get errors about not finding a CXX compiler, run this:

```
sudo xcode-select --reset
```

If you are notarizing the app locally, then add a notarization keychain to your system (APPLE_ID should be set to your Apple account, usually your email address). This only needs to be done once:

```
APPLE_ID=  
xcrun notarytool store-credentials --apple-id ${APPLE_ID}
```

When prompted, set the keychain profile to a meaningful name and enter your 10-digit organization ID as the team ID. Then enter your app-specific password. (You can get that from Apple's developer website.)

Download *wxWidgets*

```
git clone https://github.com/wxWidgets/wxWidgets.git --recurse-submodules
```

This should be cloned into a folder next to *Readability Studio*.

Build *Readability Studio*

```
git clone https://github.com/eclipse-ers/ReadabilityStudio.git \
    --recurse-submodules
cd ReadabilityStudio

cmake -S . -B ./build -DCMAKE_BUILD_TYPE=Release \
    -DHAS_MAC_TEXT_PRINTING
cmake --build ./build --target readstudio -j $(nproc) --config Release

# Uncomment to build the help manuals
#cmake --build ./build --target manuals -j $(nproc) --config Release
```

Tip

To create a debug build, pass `-DCMAKE_BUILD_TYPE=Debug` during the configuration stage, and `--config Debug` during the build stage. You can also pass `-DENABLE_SANITIZERS=TRUE` to enable AddressSanitizer support in debug builds. If you need to build with local modifications in Git submodules, add `-DALLOW_DIRTY_SUBMODULES=ON` to bypass the clean-check validation.

Code sign

💡 Tip

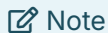
The following steps are only required if you are distributing builds outside CI, your local machine, or Eclipse's infrastructure.

If code signing locally, do the following:

(ORG_ID should be set to your 10-digit Apple developer organization ID and KEYCHAIN_PROFILE should be the keychain profile connected to your Apple ID):

```
ORG_ID=  
KEYCHAIN_PROFILE=  
  
cd installers/macos/release  
  
codesign --force --verbose=2 --options runtime \  
    --timestamp --sign ${ORG_ID} ./"Readability Studio.app"  
  
codesign --verify --verbose=2 ./"Readability Studio.app"
```

Build the DMG image



Note

If you build the “Release” version of *Readability Studio*, then the *CMake* build process will copy “Readability Studio.app” into `installers` `macos` `release`.

Go to `installers` `macos` `release` and run the following:

```
test -f ReadabilityStudio.dmg && rm ReadabilityStudio.dmg
create-dmg \
  --volname "Readability Studio Installer" \
  --volicon "../../app-logo.icns" \
  --background "../../app-logo.png" \
  --window-pos 200 120 \
  --window-size 800 400 \
  --icon-size 100 \
  --icon "readstudio.app" 200 190 \
  --hide-extension "readstudio.app" \
  --app-drop-link 600 185 \
  "ReadabilityStudio.dmg" \
  "release/"
```

If notarizing locally, then sign, notarize, staple, and verify the DMG image:

```
codesign --force --verbose=2 --sign ${ORG_ID} ./ReadabilityStudio.dmg
codesign --verify --verbose=2 ./ReadabilityStudio.dmg
hdiutil verify ./ReadabilityStudio.dmg
xcrun notarytool submit ./ReadabilityStudio.dmg \
  --keychain-profile ${KEYCHAIN_PROFILE} \
  --wait

xcrun stapler staple ./ReadabilityStudio.dmg

xcrun spctl --assess --type open --context context:primary-signature \
  --ignore-cache --verbose=2 ./ReadabilityStudio.dmg
```

If you get a notarization error, run the following (GUID will be the unique ID that the submission process just displayed):

```
xcrun notarytool log ${GUID} --keychain-profile ${KEYCHAIN_PROFILE}
```


9. Linux

Install the following tools to build *Readability Studio*:

Program

GCC 12.2.1 or higher (*gcc* and *gfortran*)

CMake 3.26 or higher (*cmake* and *cmake-extra*)

git

ApplImage (*libappimage-dev*)

R (*r-base*, optional)

Quarto (<https://quarto.org/>, optional)

POEdit (*poedit*, optional)

Install the following libraries:

Library	Debian / Ubuntu	RHEL / Fedora / openSUSE
<i>GTK 3</i>	<i>libgtk-3-dev</i>	<i>gtk3-devel</i>
<i>libcurl</i>	<i>libcurl4-openssl-dev</i>	<i>libcurl-devel</i>
<i>GStreamer MM</i>	<i>libgstreamermm-1.0-dev</i>	<i>gstreamermm-devel</i>
<i>libsecret</i>	<i>libsecret-1-dev</i>	<i>libsecret-devel</i>
<i>WebKitGTK</i>	<i>libwebkit2gtk-4.1-dev</i>	<i>webkit2gtk3-devel</i>
<i>SDL2</i>	<i>libsdl2-dev</i>	<i>SDL2-devel</i>
<i>libnotify</i>	<i>libnotify-dev</i>	<i>libnotify-devel</i>
<i>TBB</i>	<i>libtbb-dev</i>	<i>tbb-devel</i>
<i>OpenSSL</i>	<i>libssl-dev</i>	<i>openssl-devel</i>
<i>libxml2</i>	<i>libxml2-dev</i>	<i>libxml2-devel</i>
<i>gspell</i>	<i>libgspell-1-dev</i>	<i>gspell-devel</i>
<i>ImageMagick++</i> (optional)	<i>libmagick++-dev</i>	<i>ImageMagick-c++-devel</i>
<i>Poppler</i> (optional)	<i>libpoppler-cpp-dev</i>	<i>poppler-cpp-devel</i>

Download wxWidgets

```
git clone https://github.com/wxWidgets/wxWidgets.git --recurse-submodules
```

This should be cloned into a folder next to *Readability Studio*.

Build Readability Studio

```
git clone https://github.com/eclipse-ers/ReadabilityStudio.git \
    --recurse-submodules
cd ReadabilityStudio

cmake -S . -B ./build -DCMAKE_BUILD_TYPE=Release
cmake --build ./build --target readstudio -j $(nproc) --config Release

# Uncomment to build the help manuals
#cmake --build ./build --target manuals -j $(nproc) --config Release
```

💡 Tip

To create a debug build, pass `-DCMAKE_BUILD_TYPE=Debug` during the configuration stage, and `--config Debug` during the build stage. You can also pass `-DENABLE_SANITIZERS=TRUE` to enable AddressSanitizer support in debug builds.

If you need to build with local modifications in Git submodules, add `-DALLOW_DIRTY_SUBMODULES=ON` to bypass the clean-check validation.

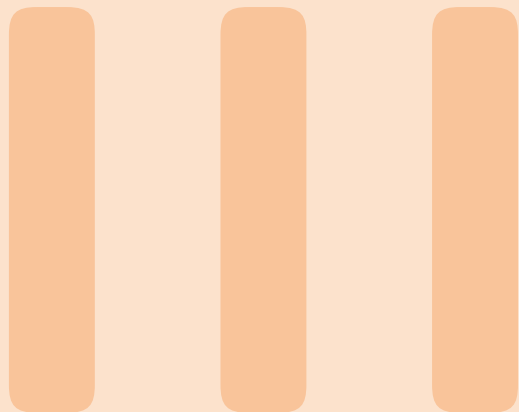
Build the AppImage

💡 Tip

The following steps are only required if you are distributing builds outside CI, your local machine, or Eclipse's infrastructure.

```
linuxdeploy-x86_64.AppImage --appdir installers/unix/AppDir \
    --executable installers/unix/AppDir/usr/bin/readstudio \
    -d installers/unix/AppDir/readstudio.desktop \
    -i installers/unix/AppDir/app-logo.svg \
    --output appimage
mv Readability_Studio*.AppImage ./installers/unix
```

Once built, the AppImage will be available in `installers` `unix`. Apply a GPG signature to it before distributing.



Contributing

10	Getting Involved	33
11	Coding Style Guide	35
12	Code Structure	39
13	Coding Procedures	41
14	Documentation	49
15	Resources	53



10. Getting Involved

All are welcome to get involved with *Readability Studio* by contributing to the code, documentation, word list, artwork, or simply by sharing feedback and ideas.

There are two main ways to get involved: reporting issues or requests and contributing changes.

Report a Bug or Request a Feature

If you encounter a problem or have an idea for improvement, please visit our issue tracker:

- Issue Tracker: <https://github.com/eclipse-ers/ReadabilityStudio/issues>

When opening an issue:

- Bug reports should clearly describe:
 - What happened
 - What you expected to happen
 - Steps to reproduce the problem (if applicable)
 - Screenshots or examples, if helpful
- Feature requests should describe:
 - The problem you're trying to solve
 - The solution you'd like to see
 - Any alternatives you've considered

To report an issue, a *GitHub* account is required. Creating a *GitHub* account is free and only takes a few minutes.

⚠ Warning

All issues are public. Please do not include sensitive or confidential information.

Contribute Code, Documentation, or Other Assets

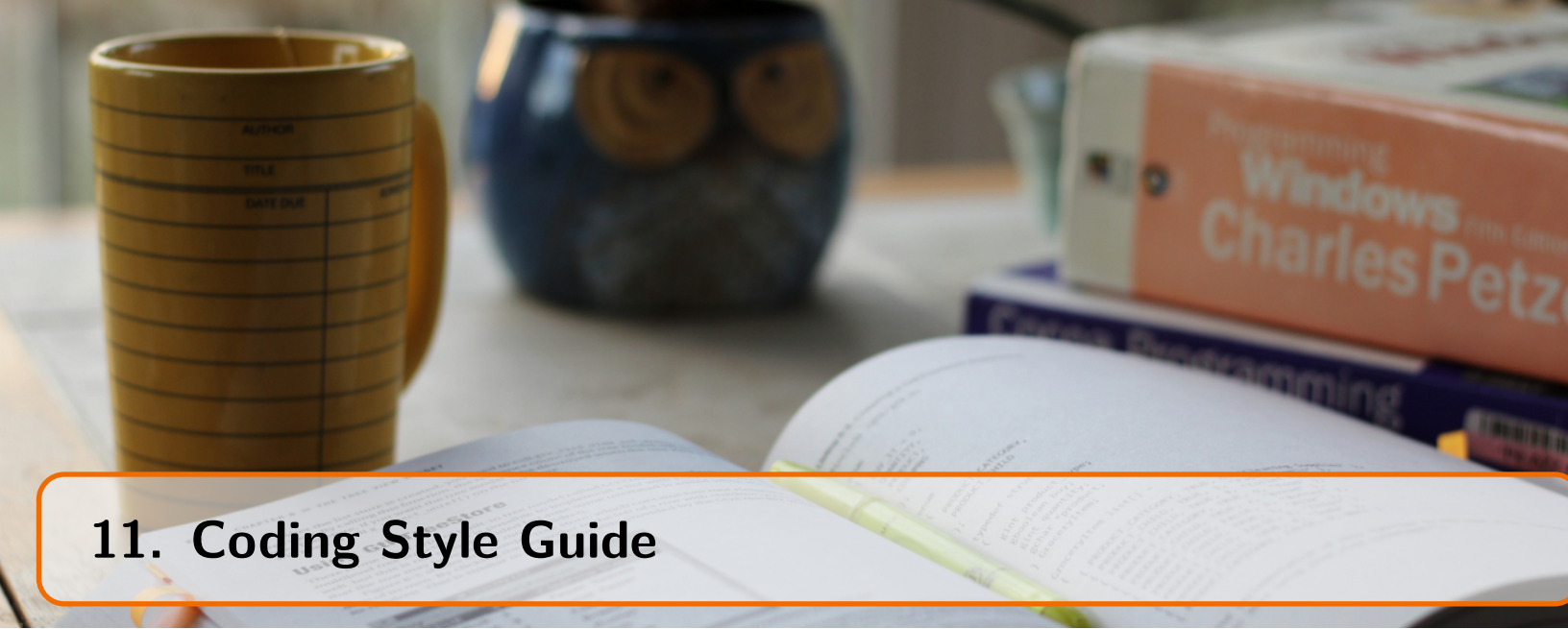
If you would like to contribute code, documentation, translations, word lists, artwork, or other project assets, follow these steps to become a contributor:

- Register with *GitHub*
Go to [GitHub.com](https://github.com) and click Sign up in the upper right corner to create a free account.
- Register with *Eclipse*
Go to [Eclipse.org](https://eclipse.org) and click Create account in the upper right corner to create a free account.
- Connect *Eclipse* and *GitHub*
Once your accounts are set up, select Edit my account in the upper right corner of *Eclipse.org*. On the My Account page, select the Account Settings tab. Enter the name of your *GitHub* account in the GitHub ID field and click Update Account.
- Digitally Sign the Eclipse ECA
While still on the My Account page, go to the Eclipse ECA tab and follow the instructions to sign the Eclipse Contributor Agreement. This certifies that your contributions are legally yours to contribute and will be made available under an open source license.

To make a contribution:

- Create a fork of our repository at <https://github.com/eclipse-ers/ReadabilityStudio.git> and make your edits to this fork.
- Submit your changes as a pull request.
- After review and approval, your contribution will be merged into the program.

While preparing your contributions, please follow our coding and technical writing style guidelines, as outlined in this chapter.



11. Coding Style Guide

Naming Conventions

The following guidelines relate to all code:

- Write in modern C++20, but only use features compatible with the latest *Visual Studio*, *GCC*, and *Apple Clang* releases
- Class member variables should be prefixed with an `m_`
- Macros should be in all caps (although macros should be avoided as much as possible)
- Global and class member constants should be in all caps
- Meaningful names should be used (only loop counters may have simple names like `i` and `j`)
- Caps should not be used in function/variable/class names, even if it's an acronym

Low-level Styles

Low-level code should follow these guidelines:

- Functions, enums, structs, classes, class member variables, and function parameters should be in `snake_case`
- Local variables should be `camelCase`

Examples of “low-level” code would be the `stemmer`, `indexing`, and `document importing` libraries.

High-level Code

High-level code (i.e., code related to user-interfaces or libraries like `wxWidgets`) should follow these guidelines:

- Functions, enums, structs, and classes should be in `PascalCase`
- Class member variables, local variables, and function parameters should be `camelCase`

Examples of “high-level” code would be the `projects`, `web harvester`, and `graphics` code.

Styling

- Use four spaces instead of tabs
- Curly braces should use Whitesmiths format:

```
if (object == nullptr)
{
    log("Memory error!");
    return false;
}
```

This applies to all blocks like if, for, while, etc.

Note

A number of these formatting styles are enforced via the project's .clang-tidy and .clang-format files. Run *clang-format* after editing any source file, and run *clang-tidy* when feasible.

Bitmask Enumerations

Bitmask enums should use left-shifting to initialize their values, as this is easier to read and maintain. For example:

```
enum DebugSettings
{
    DebugNone = 0,
    DrawBoundingBoxesOnSelection = (1 << 0),
    DrawInformationOnSelection = (1 << 1),
    DrawExtraInformation = (1 << 2),
    IncludeExperimentalCode = (1 << 3),
    AllowFileIO = (1 << 4),
    LogExtraInfo = (1 << 5)
};
```

Best Practices

- Intrinsic-type (and enumeration) member variables should be brace initialized
- Macros should be avoided when possible; prefer using functions, constexpr, etc. Macros should be limited to debugging and platform-defining sections (e.g., _WIN32).

i18n Guidelines

- The following user-facing labels and messages should be made available for translation with the `_()` (*gettext*) macro.
 - Menu and ribbon labels
 - All dialog titles and labels
 - All messages shown to the user (except log messages)
- The following should not be available for translation as they are meant for developers to review, not users:

- Log messages (e.g., `wxLogError()`)
 - Messages from low-level code (e.g., the *Excel* importer)
- Exception messages should be translatable, unless in low-level code where *gettext* mechanisms aren't available
- Strings consisting of only proper nouns (e.g., "Dale-Chall") should not be translatable
- Standard i18n guidelines apply (e.g., do not expose internal codes or numbers to translation)
- Formatting code (e.g., HTML tags) should not in translatable strings when possible



12. Code Structure

Readability Studio's code is organized in the following folder structure within `src`:

- `app`: the application and options classes
- `Catch2`: submodule containing the testing framework *Catch2* (do not edit this folder)
- `document-helpers`: document parsing helper classes
- `graphs`: readability graphs
- `indexing`: parsing, text indexing, and grammar code
- `lua`: submodule containing the *Lua* scripting language (do not edit this folder)
- `lua-scripting`: code that interfaces *Readability Studio* with *Lua*
- `OleanderStemmingLibrary`: submodule containing the *Oleander Stemming Library* (do not edit this folder)
- `projects`: standard and batch project classes
- `readability`: readability formula functions
- `results-format`: report formatting classes
- `test-helpers`: custom test parser and bundle classes
- `tinyexpr-plusplus`: submodule containing the *TinyExpr++* library.
This is the formula-parsing engine used for custom tests (do not edit this folder).
- `tinyxml2`: submodule containing the *TinyXML2* library (do not edit this folder)
- `ui`: user interface code
- `webharvester`: web harvester class
- `Wisteria-Dataviz`: submodule containing the *Wisteria-Dataviz* library.
This is the core graphics library (do not edit this folder).

Note

Although separate modules, *Wisteria-Dataviz*, *Oleander Stemming Library*, and *TinyExpr++* are all maintained by the author of *Readability Studio*. These projects can also be contributed to and are available at <https://github.com/Blake-Madden/Wisteria-Dataviz>, <https://github.com/Blake-Madden/OleanderStemmingLibrary>, and <https://github.com/Blake-Madden/tinyexpr-plusplus>, respectively.



13. Coding Procedures

Adding a New Option

Do the following to add a new option both to the program and its projects:

- Add the variable and GetXXX()/SetXXX() functions to `src/app/readability_app_options.h`
- Add an XML constant tag for aforementioned variable in `src/app/readability_app_options.h`
- Add reset logic for the variable in `ResetSettings()` (in `src/app/readability_app_options.cpp`). (This should be the same value as the initializer value in the header file.)
- Add logic to read value in `LoadOptionsFile()` (in `src/app/readability_app_options.cpp`)
- Add logic to save value in `SaveOptionsFile()` (in `src/app/readability_app_options.cpp`)

If the option is used in projects also, do the following:

- Add the variable and GetXXX()/SetXXX() functions to `src/projects/base_project.h` if a base-level value that subprojects in a batch would need. UI and graph related options should be in `src/projects/base_project_doc.h` instead.
- Initialize variable with value from `ReadabilityAppOptions` in the `BaseProject` (base-level options) or `BaseProjectDoc` constructor (`src/projects/base_project.cpp` or `src/projects/base_project_doc.cpp`)
- Add copy operation for variable in `BaseProject::CopySettings()` (base-level options) or `BaseProjectDoc::CopyDocumentLevelSettings()`
- Add reading logic to `BaseProjectDoc::LoadSettingsFile()`
- Add write logic to `BaseProjectDoc::FormatProjectSettings()`

Finally:

- Add variable to `src/ui/dialogs/tools_options_dlg.h` (for values not shown in a property grid)
 - If a value in a property grid, then add a function that returns the string for the value in the property grid; this will be used also as a lookup key
- Add variable initialization from `ReadabilityAppOptions` or project to `ToolsOptionsDlg`'s constructor

- Note that if the option is in a property grid, then there is no direct variable and instead you use a string label to get and set the values from the property grid. Instead, the default value for this field is loaded in `ToolsOptionsDlg::CreateControls()`.
- Add change watching logic for variable to `ToolsOptionsDlg::HaveXXXOptionsChanged()` if related to project settings (note there are different versions of these functions for each type of feature)
- Add saving logic to `ToolsOptionsDlg::SaveOptions()` (and related `SaveProjectXXXOptions()` function, if applicable)
- Attach variable to interface in `ToolsOptionsDlg::CreateControls()`
- Use the variable in the program where applicable
- Add it to the documentation

If wanting to edit this option from the ribbon:

- If a menu item, add it to a menu in `BaseProjectView::CreateChildFrame()` in `src/projects/base_project_view.cpp`
 - If it is a checkable menu, add logic for setting the proper check state in `BaseProjectView::Present()`
- If a ribbon button (not in the project area of the ribbon), add logic for it to be added to the ribbon in `BatchProjectView::OnItemSelected()` (`src/projects/batch_project_view.cpp`) and `ProjectView::OnItemSelected()` (`src/projects/standard_project_view.cpp`)
- If a ribbon button in the project section, add it there in `ReadabilityApp::CreateRibbon()` (`src/app/readability_app.cpp`)
- Add a `wxEVT_MENU/wxEVT_RIBBONBUTTONBAR_CLICKED` event handler for the menu/ribbon button option in `src/projects/base_project_view.h` and `src/projects/base_project_view.cpp`
 - If relevant only to standard or batch projects, then put it in the `"batch_project_view"/"standard_project_view"` files instead
- Map the event handler in `BaseProjectView`'s constructor (or `BatchProjectView`'s and `ProjectView`'s constructors)

13.1 Adding a New Option to a Graph's Ribbon

For options specific to a graph that needs to be added to the ribbon:

- If this graph doesn't have its own custom ribbon panel, then:
 - Add a `wxWindowID` constant for the graph's ribbon panel to `src/app/readability_app.h`
- Add logic for adding the new option button to the graph's panel in `ReadabilityApp::CreateRibbon()` (`src/app/readability_app.cpp`)
- Add logic for using the option to the graph's function in `src/projects/standard_project_doc.cpp` (for readability graphs, this will be its `AddXXX()` function)
- Add logic for using the new option in the graph's `DisplayXXX()` function in `src/projects/batch_project_doc.cpp`
- If not already present, create a `wxRibbonPanel*` variable from calling `hideEditPanel()` with the graph's ribbon panel ID in `BatchProjectView::OnItemSelected()` (`src/projects/batch_project_view.cpp`)

- In the same function, (if not present) add code to show the graph's panel when the selected item is of that graph's type. Also, add code to toggle the button's state (if applicable)
- Repeat the aforementioned steps, but in
 - ❏ `src/projects/standard_project_view.cpp`
- If not already present, add a function to
 - ❏ `src/projects/project_frame.cpp` to handle the new ribbon button being clicked
 Refer to `ProjectDocChildFrame::OnShowcaseKeyItems()` as an example

Adding a New Test

- Add the actual function to ❏ `src/readability/english_readability.h` (or other file if a different language). (This only applies to linear-regression based tests with a formula, not tests that calculate their values graphically [e.g., Fry].)
- Add its definition to `BaseProject::InitializeStandardReadabilityTests()` (or subfunction)
- Add internal name constant to ❏ `src/results_format/readability_messages.h`. This will be a string ID that is used throughout the program, documentation, and resources to identify the test.
- Add its `AddXXXTest()` function to ❏ `src/projects/base_project.cpp`
- If a graphical test (e.g., Fry), then also do the following:
 - If the graph calculates the score (e.g., Fry), then the `AddXXXTest()` function must be virtual and derived in ❏ `src/projects/standard_project_doc.h` also
 - Otherwise, if the score is calculated by a formula and the graph is a supplement to visualize that score (e.g., Crawford), then add a virtual `AddXXXGraph()` function to ❏ `src/projects/base_project.h` and call that from the test's `AddXXXTest()` function in ❏ `src/projects/base_project.cpp`
 - Handle its exclusion in `ProjectDoc::DisplayReadabilityGraphs()`
 - Add its `GetXXXLabel()` function in ❏ `src/projects/base_project_view.h`
 - Define its graph window ID in ❏ `src/projects/base_project_view.h`
 - Add aforementioned constant to `m_dynamicIdMap` in ❏ `src/app/readability_app.cpp`, mapping it to an unused numeric ID
 - Add aforementioned numeric ID and constant name into the `GraphType` enum in ❏ `resources/scripting/rs-constants.lua`
 - Handle the aforementioned window ID in `ProjectView::OnEditGraphOptions()`, `ProjectView::OnHyperlinkClicked()`, `ProjectView::OnItemSelected()`
 - Load results in `BatchProjectDoc::LoadScoresSection()` if graph handles its own scoring (e.g., Fry)
 - If the graph calculates the score, then handle loading its scores into the graph and toggling its shown state in `BatchProjectDoc::DisplayReadabilityGraphs()` (or a subfunction from that)
 - If the graph is a supplement to a formula, then handle calculating its scores and toggling its shown state in `BatchProjectDoc::DisplayReadabilityGraphs()` (or a subfunction from that)
 - Add `Set/GetXXXGraph()` functions to ❏ `src/projects/batch_project_view.h`
 - Show/hide window in `BatchProjectDoc::DisplayScores()`
 - Add its icon loading to `ReadabilityApp::InitProjectSidebar()`

- Connect the sidebar item to the icon in `ProjectView::UpdateSideBarIcons()` and `BatchProjectView::UpdateSideBarIcons()`
- Add a blank graph for it in `MainFrame::OnBlankGraph()` and `MainFrame::FillBlankGraphsMenu()`
- Bind the XRC ID for the test to `MainFrame::OnBlankGraph()` in `MainFrame::MainFrame()`
- Add this `AddXXXTest()` function to the test function pointer map in `BaseProject::BaseProject()`
- If an index test, add its score formatting logic to `BatchProjectView::UpdateStatAndTestPanels()`
- Document it in the help:
 - Add a topic for it in `docs/readability-studio-manual/[language]`, where “[language]” is the language that the test is meant for
 - Add it to the proper dataset in `docs/readability-studio-manual/data` (e.g., “EnglishTests.txt”)
 - If a graphical test, automate making a screenshot of the graph for the help
- Add an icon to `resources/images/tests` in SVG format using the test’s string ID

Adding a New Familiar Word List

The following documents how to add a familiar word list (used by a test, such as Dale-Chall) to the program. These are extra steps in addendum to adding a new test.

Note

This applies to full words lists (e.g., Dale-Chall), not supplements such as Stocker’s Catholic word list.

- Create a word list file and add to the resources (`resources/words/word-lists`). (This should be a single column text file.)
- Add a `word_list` object to `BaseProject` (in `src/projects/base_project.h`)
- Load the text file into aforementioned `word_list` in `ReadabilityApp::LoadWordLists()`
- Add a check that the list is sorted in `ReadabilityApp::VerifyWordLists()`

Adding Statistics Fields

- Add unique and total unfamiliar word statistic fields to `BaseProject`
- Add XML constants in `src/app/readability_app_options.h` and `src/app/readability_app_options.cpp` for these unique and total values
- Add `GetXXX()/SetXXX()` functions in `BaseProject` for these fields
- Zero out these values in `BaseProject`’s constructor and initializers
- Zero out these values in `BaseProject::LoadHardWords()`
- In `BaseProject::LoadHardWords()`, create an `is_familiar_word` object connected up to the word list. (An example is the `isDCWord` object.)
 - Use it in this function to tabulate the number of total and unique unfamiliar words.
- Format and display these values in `BaseProject::FormatStatisticsInfo()`
- Load these values in `ProjectDoc::LoadMetaFile()`
- Save these values in `ProjectDoc::OnSaveDocument()`
- Add unfamiliar label function to `src/projects/base_project_view.h`

Adding Support for the List Window

- Add a page ID for the list in `src/projects/base_project_view.h`
- Add this ID to the `ListType` enumeration in `resources/scripting/rs-constants.lua`
- Add hard word list data provider to `BaseProject`
- Initialize the data provider in `BaseProject`'s constructor
- Delete the data provider in `BaseProject`'s destructor
- In `BaseProject::LoadHardWords()`, delete all the items in the data provider, and then resize it to the size of the word map
- In the aforementioned unfamiliar word handler in `BaseProject::LoadHardWords()` (e.g., `isDCWord`), fill the data provider with the unfamiliar words
- In `ProjectDoc::DisplayWordsBreakdown()`, add logic to add a page for the data provider and show it in a list control. (Should involved showing or hiding these windows if the test is included.)
- Add `EVT_LIST_ITEM_ACTIVATED` message mapping for list ID to `ProjectView::OnListDbClick()`
- Add handler for ID to jump to the respective highlighted text window in `ProjectView::OnListDbClick()` (see following instructions for highlighted test window)

Adding Support for the Text Window

- Add a page ID for the text window in `src/projects/base_project_view.h`
- Add this ID to the `HighlightedReportType` enumeration in `resources/scripting/rs-constants.lua`
- Add a `wxFormattedTextCtrl` window in `src/projects/standard_project_doc.h`
- Add unfamiliar function object (based on `IsNotFamiliarWordWithHighlighting`) to `ProjectDoc::DisplayHighlightedText()`. (An example is `isNotDCWord`.)
- Add a legend line for the word list in text window in `ProjectDoc::DisplayHighlightedText()`
- Add the legend's line length to `maxLegendSize`
- Add a legend (including the aforementioned legend line)
- Load and fill the text window in `ProjectDoc::DisplayHighlightedText()`
- Hide or show the text window in `ProjectDoc::DisplayHighlightedText()` based on whether its test is included in the project
- Also hide or show the text window in `ProjectDoc::UpdateHighlightedTextWindows()`

Remaining Standard Project Steps

- Add hyperlink handler (from formatted stats) to `ProjectView::OnHyperlinkClicked()`
- Add bar for unfamiliar words to bar chart in `ProjectDoc::DisplayWordCharts()`
- Add cases for both the list and text page IDs in `MainFrame::OnHelpContents()` to go to the help topic "reviewing-word-breakdowns.html"

Batch Project

- Increase the size of the hard word list data provider in `BatchProjectDoc::LoadHardWords()`
- Add word stats into data provider in `BatchProjectDoc::LoadHardWords()`
- Insert columns into hard word list in `BatchProjectDoc::DisplayHardWords()`

Custom Test Support (Low-level)

- Add word list to `custom_test` (`src/readability/custom_readability_test.h`) class using an `is_familiar_word` object

- Add flag for including that list, along with getter and setter functions
 - These should be initialized to false
- In `custom_test`'s constructor, set the `is_familiar_word` object to use the test's prescribed proper noun and number logic
- Add logic for `operator==`, `is_using_familiar_words()`, `reset()`, `is_word_familiar_any_included_list()`, `is_word_familiar_all_included_list()`

Custom Test Support (Higher-level)

- Add XML string constant for saving word list flag for custom tests to `src/app/readability_app_options.h` (e.g., `XML_INCLUDE_HARRIS_JACOBSON_LIST`)
- Update any calls to `BaseProjectDoc::AddGlobalCustomReadabilityTest()` to include the word list and respective flag (the `CustomReadabilityTest` typedef passed to this will need these passed to its constructor)
- Add save and read logic for flag in `src/projects/base_project_doc.cpp`
- Add save and read logic for settings file (`ReadabilityAppOptions::LoadOptionsFile()` and `ReadabilityAppOptions::SaveOptionsFile()`)
- Add logic to `ShowCustomTestDialogFamiliarWords()` and `SetCustomTestDialogIncludedLists()` in `src/luascripting/luascreenshot.cpp`
- Set and read this flag from `EditCustomTest` in `src/app/readability_app.cpp`
- Add logic to `OnAddCustomTest()` in `src/app/readability_app.cpp`
- In `AddCustomReadabilityTest()` (in `src/projects/base_project.cpp`), mention the inclusion of this list in the test's description

Custom Test Dialog

- Add flag to custom test dialog
- Add the flag to the `UpdateOptions()` function
- Initialize the flag to false
- Add a checkbox ID for the flag
- Add a checkbox to the custom test dialog
- Add a message mapping for the new checkbox ID
- Update calls in `OnAddCustomTest()` and `OnEditCustomTest()` to set and get the flags to and from this dialog

Custom Test Formula

- Add a `GetCustomXXXSignature()` function in `src/test_helpers/readability_formula_parser.h` that returns the label "`CustomXXX()`" (where XXX is the name of the word-list test)
- Add a static double `CustomXXX(const te_expr* context)` function in `src/test_helpers/readability_formula_parser.cpp` that handles a custom version of the original test
This function is what gets mapped to "`CustomXXX()`." (Refer to `CustomNewDaleChall()` as an example.)
- Map this function in `ReadabilityFormulaParser::ReadabilityFormulaParser()`
- Add the new `CustomXXX()` function to `m_customFamiliarWords` in `CustomTestDlg::Create()`
- Add handling logic for this function to `CustomTestDlg::ValidateFormula()`
- Add a menu option to create a new test based on `CustomXXX()`

- Add a menu event mapping for adding a new test using this new formula in `src/app/readability_app.cpp`
- Add handling logic to `MainFrame::OnAddCustomTest()`
- If the original word test has custom logic for how to handle text exclusion:
 - Add a bit for it in `CustomReadabilityTestInterface` in `src/projects/base_project.h`
 - Handle the text exclusion logic in `BaseProject::LoadHardWords()`
 - In the custom test dialog, warn the user if they select exclusion/proper noun/number options that differ from the test

Word List Dialog

- Add page and list IDs for the word list to `src/dialogs/word_list_dlg.h`
- Handle the list being selected in `WordListDlg::GetActiveList()`
- Create the list by calling `AddSingleColumnPage()` in `WordListDlg::CreateControls()`

Note

If it is a more complex word list (with categories, like Dolch), then that will need its own logic.

- Add a menu ID for the word list and map it to `MainFrame::OnWordListByPage` (`src/app/readability_app.cpp`)
- Add the word list to the menu in `MainFrame::FillMenuWithWordList()`
- Handle the menu option for the word list in `MainFrame::OnWordListByPage()`

Adding a New Custom-Test Formula Function

- Add a static function wrapper to the function you wish to call from the program in `src/test_helpers/readability_formula_parser.cpp` (e.g., `HardFogWordCount`)
- Map the function (and shortcut, if applicable) in `ReadabilityFormulaParser::ReadabilityFormulaParser()`
- Document the function in `CustomTestDlg::Create()`
- Document the function in `docs/readability-studio-manual/custom-tests-and-bundles.qmd`

Adding a New Stipple Shape

When a new stipple shape is added to the `Wisteria-Dataviz` submodule, it can be included in this program.

- Add a mapping of the displayable name and internal *Wisteria* key in `m_shapeMap` in `src/app/readability_app.cpp`
- In the API manual (`readability-studio-api`), add the new shape to the “`SetStippleShape.qmd`” topics. (This will need to be the internal keys, not the displayable name.)

Adding a New Color Scheme

When a new color scheme is added to the `Wisteria-Dataviz` submodule, it can be included in this program.

- Add a mapping of the displayable name and internal *Wisteria* key in `m_colorSchemeMap` in `src/app/readability_app.cpp`. (This will need to be the internal keys, not the displayable name.)

13.2 Adding New Files

If adding a new source file, image, or word list to the build, run the configuration step for the main “CMakeLists.txt.” This will re-generate the following files, which the build script will include:

- `cmake/includes/files.cmake`: the source files built into the program (including files from submodules)
- `cmake/includes/images.cmake`: the images to be packaged into “res.wad” (the distributable resource file)
- `cmake/includes/words.cmake`: the word lists to be packaged into “res.wad”

The file `cmake/list-files.cmake` gathers and filters the files that should be included. If you need to filter more files, edit the regular expressions in this script.



14. Documentation

Style Guide

The following styles should be applied to all help projects:

- Software and library names should be italicized
- References to dialog/menu/button/tab names should be in bold
- File and folder names should be double quoted
- File paths should be put in `file_path` blocks
- Data types (programming docs) should be in typewriter font
- Parameters to functions (programming docs) should be italicized
- Values (numbers and strings) in programming docs should be in typewriter font
- Values elsewhere:
 - Numbers should not be formatted
 - Strings and single characters should be italicized (quoting them makes them difficult to read)

Editing

The documentation is stored in *Quarto* projects in the `readability-studio-manual`, `readability-studio-api`, and `sysadmin` folders.

To change any of these projects' settings or add topics to them, edit these files:

- `readability-studio-manual`:
 - `"_quarto.yml.in"`: the base settings
 - `"_quarto-manual.yml"`: the settings specific to the $\text{\LaTeX}$ manual
 - `"_quarto-online.yml"`: the settings specific to the HTML help
 - `"_quarto-sysadmin.yml"`: the `sysadmin` project file
- `sysadmin`: see the `"_quarto-sysadmin.yml"` file in `readability-studio-manual`
- `readability-studio-api`: `"_quarto.yml"`

⚠ Warning

Do not edit `docs/readability-studio-manual/_quarto.yml`, as it is generated by the help build script. Edit `docs/readability-studio-manual/_quarto.yml.in` instead if needing to make changes.

⚠ Warning

Do not edit any topics in `readability-studio-api/enums`. These files are generated by the help build script.

⚠ Warning

The project folders `readability-test-reference`, `release-notes`, and `shortcuts-cheatsheet` are built from content in the `readability-studio-manual` folder. They are dynamically built by the build script and should not be edited.

Screenshots

Generating Screenshots

All screenshots are generated from a script; they should never be created manually.

To generate screenshots, run the script `docs/generate-screenshots.lua` from *Readability Studio*. This will generate all new screenshots in the folder `docs/readability-studio-manual/images`.

The subdirectory `non-generated` will not be affected, but does not need to be. The `non-generated` folder should not contain images affected by the program's UI changes. It also should not contain images with content that should be localized.

Reviewing Screenshots

After generating new screenshots, build the help and review the following topics to verify that the values referenced in the screenshots match the help:

- "reviewing-readability-graphs.html"
- "reviewing-statistics.html"
- "reviewing-batch-scores.html"
- "reviewing-batch-box-plots.html"
- "reviewing-batch-histograms.html"
- "searching-for-overly-long-sentences.html"
- "reviewing-dolch.html"
- "analyzing-a-work-in-progress.html"
- "reviewing-a-collection-of-documents.html"
- "excluding-text-from-analysis.html"
- "creating-a-custom-index-test.html"
- "finding-difficult-sections.html"

Building

The following software is required to build the manuals:

- *R* (CRAN)
- *Quarto*
- $\text{\LaTeX}$ (will usually be installed with *Quarto*)

Run `docs/build-help-projects.R` to build all help projects. This should install all needed  and $\text{\LaTeX}$ packages the first time you run it. If there are any errors while installing the $\text{\LaTeX}$ packages, run the script again; you may experience connection issues to *ctan* and re-running the script will fix it.



15. Resources

Art Assets

All art assets are stored in `resources/images`. Images should be saved into the appropriate folder in SVG format (except the `splash` folder, these are JPEGs). SVGs should have an aspect ratio with the same width and height.

Application Icon

If the application icon (`resources/images/ribbon/app-logo.svg`) is changed, then in the root directory:

- Save a copy as “app-logo.png” (1024×1024 resolution)
- Save a copy as “app-logo.ico” (512×512 resolution)
- On macOS, run this in a terminal:

```
mkdir app-logo.iconset
sips -z 16 16      app-logo.png --out app-logo.iconset/icon_16x16.png
sips -z 32 32      app-logo.png --out app-logo.iconset/icon_16x16@2x.png
sips -z 32 32      app-logo.png --out app-logo.iconset/icon_32x32.png
sips -z 64 64      app-logo.png --out app-logo.iconset/icon_32x32@2x.png
sips -z 128 128     app-logo.png --out app-logo.iconset/icon_128x128.png
sips -z 256 256     app-logo.png --out app-logo.iconset/icon_128x128@2x.png
sips -z 256 256     app-logo.png --out app-logo.iconset/icon_256x256.png
sips -z 512 512     app-logo.png --out app-logo.iconset/icon_256x256@2x.png
sips -z 512 512     app-logo.png --out app-logo.iconset/icon_512x512.png
cp app-logo.png app-logo.iconset/icon_512x512@2x.png
iconutil -c icns app-logo.iconset
rm -R app-logo.iconset
```

Word Lists

Editing Lists

In the  resources/words folder, the following word lists are used by the program. Some can be edited, others are generated. Also, all these files are meant for all languages, unless otherwise noted.

Please refer to the following file & folder breakdown for reference:

- "base-english-dictionary.txt": main English dictionary. All forms of a given word (e.g., plural, possessive) should go in here. Note that proper nouns should not go in here
- "base-non-personal.txt": places (e.g., cities, countries), company names, species names, etc. should go in here
- "base-personal.txt": person names should go in there
-  abbreviations:
 - "abbreviations.txt": list of abbreviations that the program uses to determine abbreviations. (Words in here must include their periods.)
 - "non-abbreviations.txt": list of words that should never been seen as abbreviations. This will be long words that contain no vowels, but are still an actual word.
-  articles:
 - "a-exceptions.txt": these are words that are correct to follow an a that the program would otherwise misclassify
 - "an-exceptions.txt": these are words that are correct to follow an an that the program would otherwise misclassify
-  citation-headers:
 - "citations.txt": list of phrases that would mark the beginning of a citation section in a document
-  copyright-notices:
 - "notices.txt": list of phrases that would mark a copyright notice in a document
-  dictionaries: all files in this folder are generated, do not edit anything in here
-  past-participles:
 - "exceptions.txt": adjectives that can be mistaken for past-tense verbs.
These are entered as overrides that the program would see as passive voice otherwise.
-  programming:
 - "all-languages.txt": this file is generated, do not edit
 - All other files in here are keywords/function names/etc. for various programming languages.
These are used by the spell checker if the user is ignoring programming words.
-  proper-nouns: all files in this folder are generated, do not edit anything in here
-  stop-words:
 - "english.txt": common English words that should be excluded from the list of key words
 - "german.txt": common German words that should be excluded from the list of key words
 - "proper-nouns-stoplist.txt": common words that would never be proper. This is used to prevent uppercased words from being seen as proper when they really aren't. (This should include stop words from all languages.)
 - "spanish.txt": common Spanish words that should be excluded from the list of key words
-  word-lists:
 - "dolch.txt": Dolch list of Sight Words
 - "harris-jacobson.txt": Harris-Jacobson known words

- “new-dale-chall.txt”: New Dale-Chall known words
- “revised-spache.txt”: Revised Spache known words
- “stocker-catholic-supplement.txt”: Stocker’s Catholic Supplement (used with Dale-Chall)
-  wordy-phrases:
 - “base-english.txt”: English phrases
 - “base-german.txt”: German phrases
 - “base-spanish.txt”: Spanish phrases
 - “common-errors.txt”: Phrases for all languages
 - “dale-chall-replacements.txt”: generated, do not edit.
 - “english.txt”: generated, do not edit
 - “german.txt”: generated, do not edit
 - “harris-jacobson-replacements.txt”: generated, do not edit
 - “single-word-replacements-english.txt”: generated, do not edit
 - “spache-replacements.txt”: generated, do not edit
 - “spanish.txt”: generated, do not edit

For phrase files (in the  wordy-phrases folder), each row should be a phrase, and the column values (tab-delimited) are:

- Phrase
- Suggestion (optional)
- Type (wordy = 0, redundant = 1, cliché = 2, error = 3)
- Word preceding the phrase that will override the grammar checker (optional, can be multiple values delimited by ; or ,)
- Word following the phrase that will override the grammar checker (optional, can be multiple values delimited by ; or ,)

Finalizing Word Lists

After making changes to a word list, run the script  resources/finalize-word-lists.lua from *Readability Studio*. This will sort, merge, remove duplicates, and generate supplemental word lists.

Translations

All translation catalogs (*.po files) are in the  locale folder. After editing a catalog, compile it with a *gettext* tool (e.g., *POEdit*) into a *.mo file.

For the translations provided with *wxWidgets*:

- compile the required *.po files
- rename the output to “wxstd.mo”
- copy it into a folder with the language’s canonical name in  locale.

For example, if compiling  wxWidgets/locale/es.po, then copy the output “es.mo” to  locale/es/wxstd.mo.

The next time that the *CMake* build process is run, all compiled *.mo files will be copied into the build and installer folders.

Windows Installer

The *Windows* installer project is located in `installers/windows`. This is an *Inno Setup* project that contains the following structure:

- "readstudio.iss": the main project file
- "setup.isl.in": variables from the *CMake* script that the installer may use (e.g., product version)
- "lang.isl": the list of translation catalogs
- `messages`: this folder contains the translation catalogs listed in "lang.isl"

ABOUT THE AUTHOR

BLAKE MADDEN is the Director of Institutional Research at Edison State Community College. He earned a Bachelor of Arts in English from the University of Dayton and has researched readability formulas for more than twenty years. Before entering academia, he spent eighteen years working in the statistical software industry, primarily in localization engineering.



COLOPHON



This book was typeset using $\text{\LaTeX}$ and built with open-source tools.

DESIGN

Cover design by Blake Madden.

Interior layout based on the *Legrand Orange Book*.

A TYPOGRAPHY

Cover	— Playfair Display <i>by Claus Eggers Sørensen</i>
Body	— Inter <i>by Rasmus Andersson</i>
Code Snippets	— Inconsolata <i>by Raph Levien</i>
Japanese Content	— Noto Serif JP <i>by Google</i>

PHOTOGRAPHY

Cover and interior photographs by the author and from the *Images from the History of Medicine* (IHM) collection, National Library of Medicine.